CS305 2025 Fall Assignment 1 - Local DNS Server

A **local DNS server** is a server which is setup inside of a company, homes network, or residential ISP for address/name translations. The goal of this assignment is to implement a **local DNS server** with several functionalities in Python.

IMPOTANT NOTES:

- Your python code should be named as "LocalDNSServer.py" using UTF-8 as file encoding.
- The local DNS server should listen on **port 5533** (do NOT change it!!!) and **Your IP address for outbound communication**
- When querying, the RD in DNS message should be set to **0** (or set the flag to be 0x0000), which means that your DNS query is iterative.
- You need to submit your python code and a report for this assignment. Your report must include your code **comments**, **explanations** of the code, and the **test results** for each task. If your report fails to fully reflect your design, we may deduct points accordingly.
- A basic template will be provided and you need to implement the functionalities in the **TODO** fields. You may choose to not use the template, but you need to confirm your solutions match the corresponding input and output as shown in the examples below.
- We will use dig as the client in this assignment.

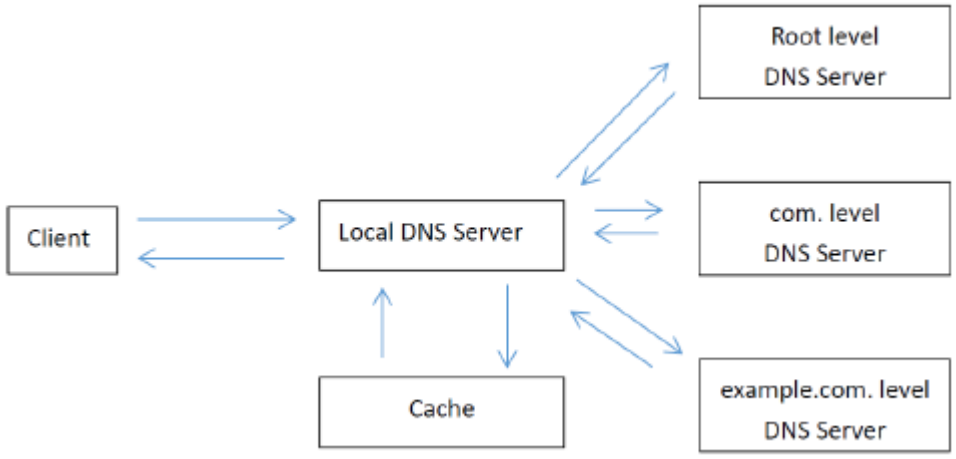
Environments:

- Python 3.12.10
- `dnspython` and `dnslib` libraries (optional, but recommended and used in the template)
- You should not use libraries other than `dnslib`, `dnspython`, and Python's native libraries.

Tasks

Task 1: Iterative Query (50 points)

Your Local DNS server should listen and accept DNS queries from client, make iterative query and reply the request.



This task includes:

1. Find and listen to your **IP address for outbound communication** for DNS query.
2. Receive and analyze DNS queries
3. Make iterative query to different levels of DNS server.
4. Reply to client with the response.

Take a DNS query to "www.example.com" as an example what the iterative query is:

- When server starts, the local DNS server queries the addresses of the **root level DNS server** from a **public DNS server**.
- The Local DNS server receives a query about "www.example.com"
- The Local DNS server resolve this request iteratively:
    1. The local DNS server sends the query to **root level DNS Server**, and gets a DNS response message about the list of **.com level** DNS servers.
    2. The local DNS server sends the query to **.com level**, and gets a DNS response message about the list of **.example.com level** DNS servers.
    3. The local DNS server then sends the query to **.example.com** level DNS Server, and gets the DNS response message about the mapping of the original DNS query.
- The local DNS server replies to the client.

There will be **5 given queries** for this task. The given queries will be included in `test.py` file. If your DNS server can handle these queries, you can get full points from this task.

Task 2: Caching (25 points)

To improve the efficiency of the local DNS server, caching is an important technique. Your local DNS server should be able to cache historical queries and reply cached results to improve performance. This task includes:

1. Load cache file from disk when server starts. (5 points)
2. Write cache file to disk when server stops. (5 points)
3. When receiving a DNS request, reply cached result if cache hits. (5 points)
4. When receiving a DNS request, make DNS query and update cache when cache miss or outdated. (5 points)
5. Maintain a TTL for cache. For a NXDOMAIN reply (domain not found), the TTL should be **60s**. Otherwise, the TTL should be **300s**. (5 points)

Task 3: Optional Functionalities (25 points)

The local DNS server should be able to handle diverse requests and concurrent requests. Also, there are some optional but useful functionalities for local DNS server. Two of them are DNS redirection and DNS filtering. Here is some optional functionalities you can add to your DNS server:

1. Manage concurrent requests. If your DNS server needs to wait for the previous request to complete before accepting the next one, it will be inefficient. You may use **Threading** or **Async IO** to handle concurrent requests. (10 points)
2. DNS redirection. The local DNS server can redirect a DNS request to a different IP address. For example, redirecting the following domains:
    - `google.com`, `www.google.com` to `127.0.0.1`
    - `doubleclick.net`, `www.google-analytics.com` to `0.0.0.0`
    - `friendly.name` to `8.8.8.8`

    (5 points)
3. DNS filter. The local DNS server can filter domains we don't want to reply, and reply a `REFUSED` packet to the client (`RCODE=5`). For example, blocking the following domains:
    - `malware-site.com`
    - `phishing-attack.net`
    - `ads.annoying-tracker.net`
    - `stats.unwanted-data-miner.org`
    - `distracting-social-media.com`

    (5 points)
4. Extents to 5 given queries in **task 1**, we will test 20 more queries for your DNS server. If your server can handle these requests, you can get this points. Your score will be proportional to the test you completed (10 points)
5. To test the efficiency of your DNS server, we will make concurrent requests. We will send all 20 requests for three times, to make sure your server can cache corresponding results. If your server can respond to

all the requests in **1 second** in the last test, you can get this points. Also, the score will be proportional to the test you completed. (10 points)

6. Other functionalities. If your DNS server implements functions beyond its intended purpose, please describe the specific role and implementation method of this function in the report. We will assign points based on the difficulty of implementation and the function's practical impact.

The total score for this assignment will **NOT exceed 100**.

Testing

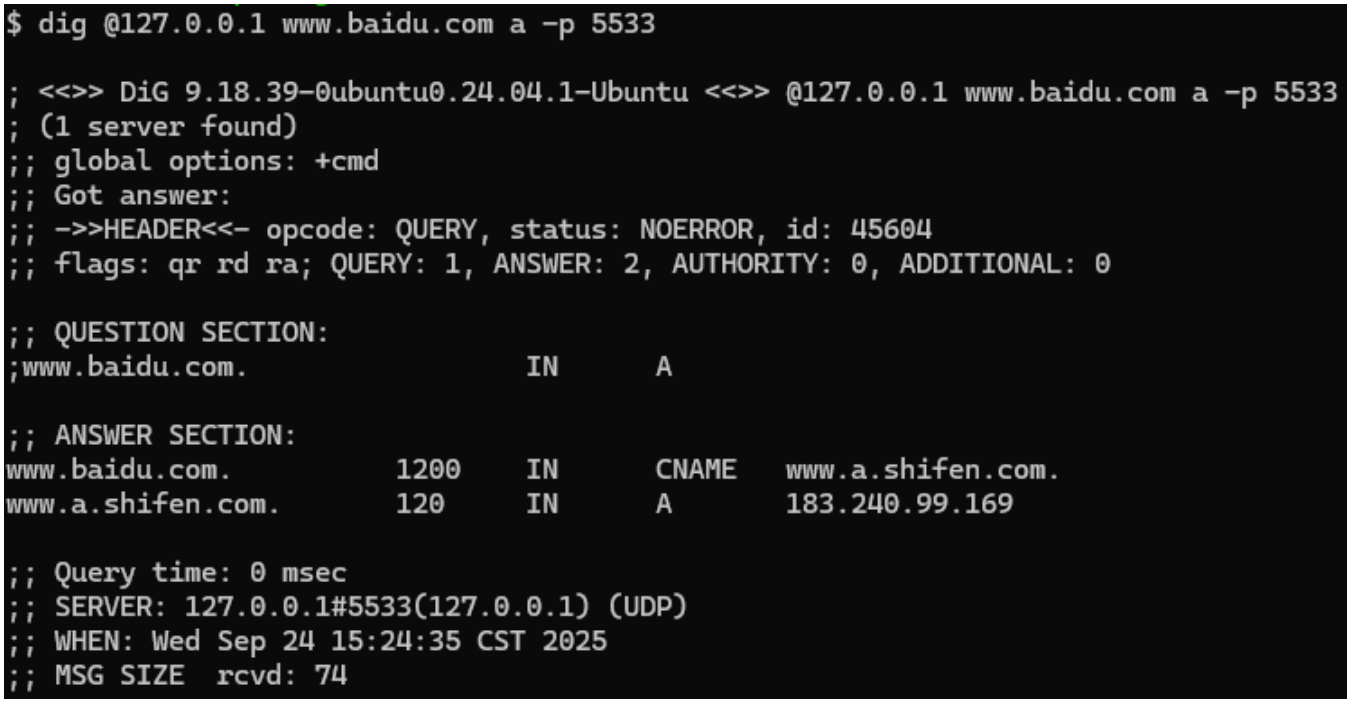
There are multiple ways to test your DNS server. We recommend you to use `dig` tool for single tests, and our `test.py` script for multiple tests. You can also use wireshark to capture your DNS requests.

Dig tool

You can run this `dig` command in your command line interface to create an A record DNS request:

```
dig @"your ipv4 address" "queried domain name" a -p "your port"
```

Then, your console should print your local DNS server's response.



If you resolve a domain name and get a result in CNAME type, you should keep that result and include it in the response

The above output example shows:

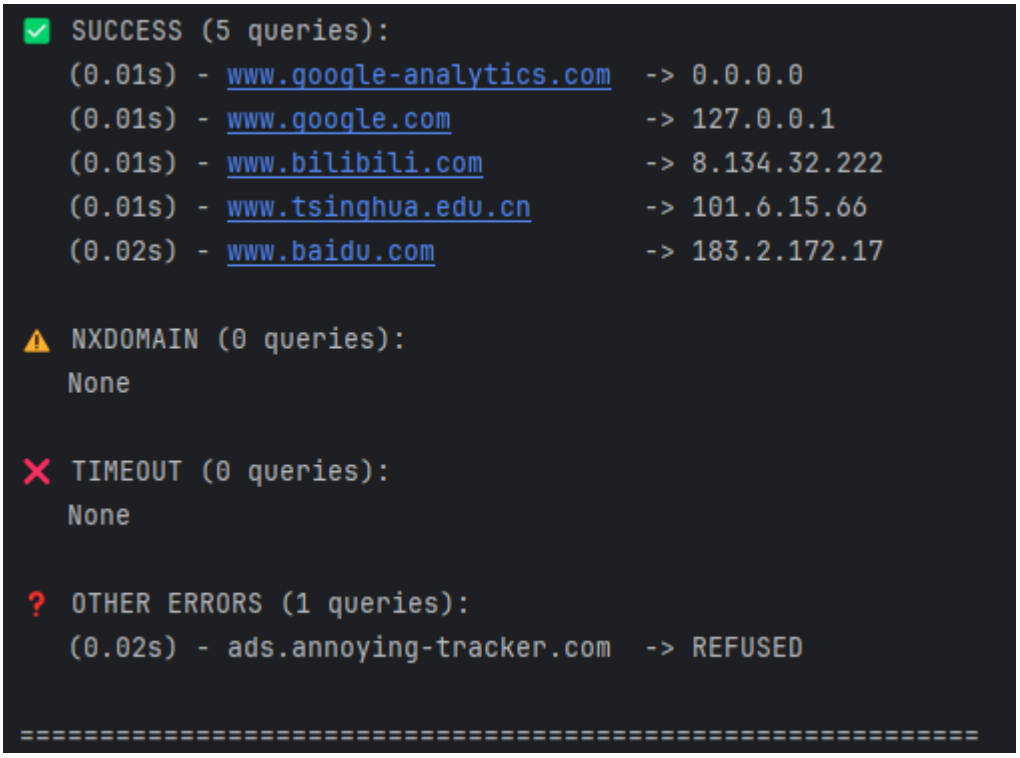
1. when you search www.baidu.com and find a CNAME type record of www.a.shifen.com.
2. Then use www.a.shifen.com to query the final response, which is shown as the above screenshot.

Test script

We will provide a `test.py` script to test your server. You can modify the DOMAINS_TO_TEST list to change the domains to test.

```
DOMAINS_TO_TEST = [
    "www.google.com", "www.github.com", "www.sustech.edu.cn", "www.baidu.com", "malware.example.com"
]
```

The test script will automatically run multiple `dig` commands to send requests to local server, print each result, and print a test summary:



What to submit

When submitting, you need to submit a report along with all the code, packed into a .zip archive named `<sid>_DNSServer.zip`, where `<sid>` is your student ID. There are some note to the submission:

- You should provide your implementation in the file `LocalDNSServer.py`, and other files if needed. The cache file will be deleted when testing.
- Please provide comments in your code to explain different parts of your code. You need to at least illustrate in your implementation about each task you complete.

Report

The report should explain your code and result. For each task, you need to include the following contents:

1. Your implementation and explanation. It should contain your code for each task. If your report does not include an explanation for part of the tasks, points will be deducted.
2. Test screenshots. Your test screenshots need to include the results of the `test.py` **script** and the

Wireshark capture, to demonstrate that your code can complete this task. If your report does not include a test result for part of the tasks, points will be deducted.

3. Your report may also include any other content that you believe can better describe your work.

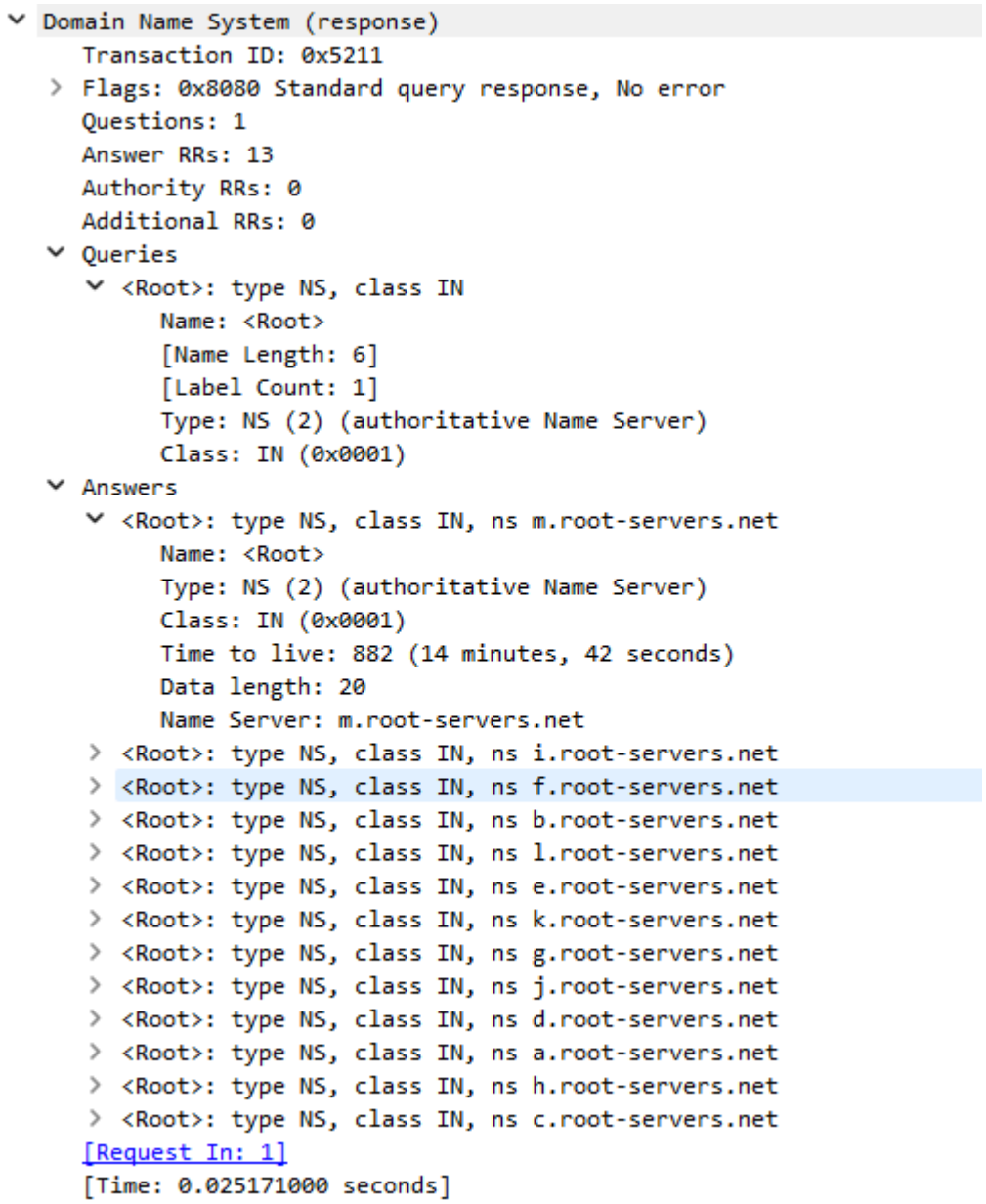
Tips

The below WireShark screenshots show the process about resolving a DNS query **iteratively**. It follows the step of iterative query in task 1.

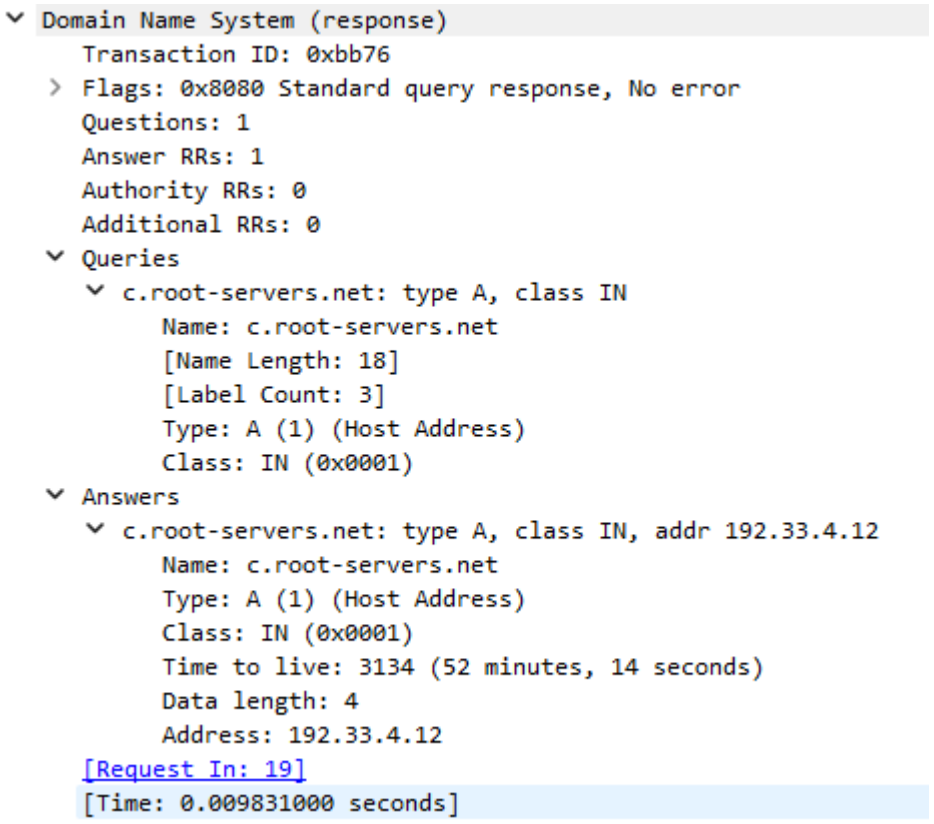
1. Initialization

When your server initializes, you need to query the IP address for root DNS server. This IP address is the starting point for iterative query

① query public DNS 223.5.5.5 and get list of root DNS server in NS type



② query public DNS 223.5.5.5 and get IP address of root DNS server



2. Query for domain

When your server receives a request, do a query iteratively. For different domain the query process may be different. This is an example for querying `www.baidu.com`

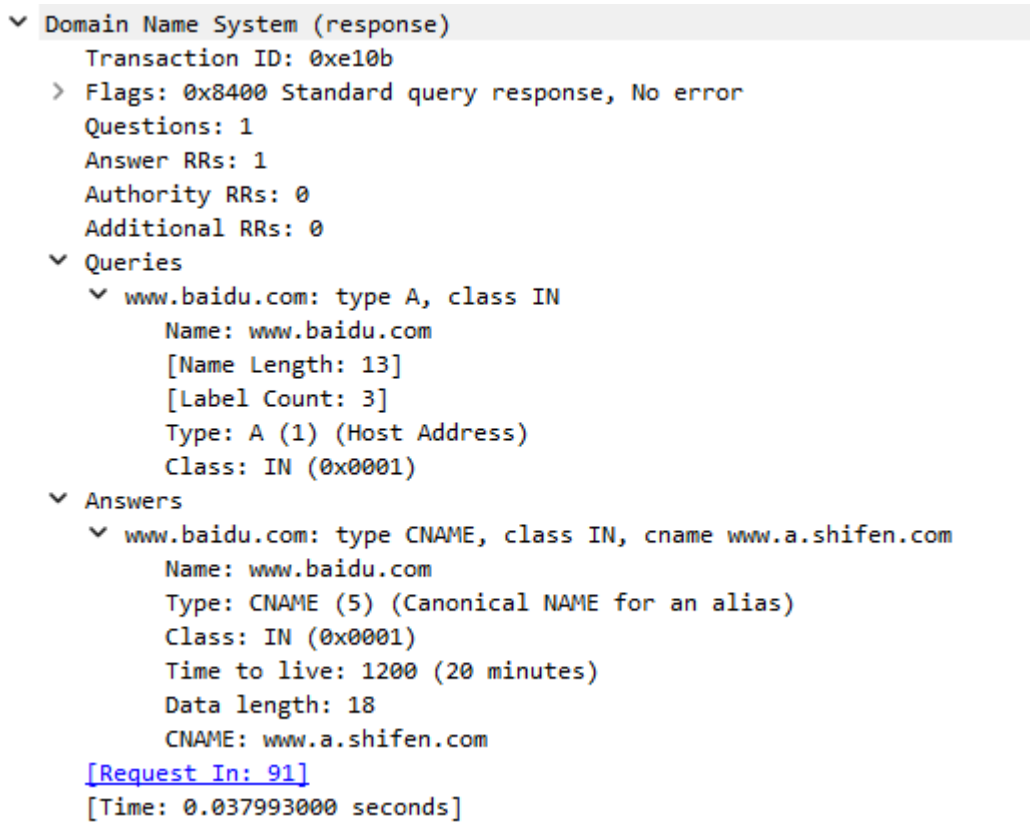
③ query root DNS server and get IP address of top-level DNS server

```
Domain Name System (response)
    Transaction ID: 0x6d03
  > Flags: 0x8000 Standard query response, No error
    Questions: 1
    Answer RRs: 0
    Authority RRs: 13
    Additional RRs: 14
  v Queries
    v www.baidu.com: type A, class IN
        Name: www.baidu.com
        [Name Length: 13]
        [Label Count: 3]
        Type: A (1) (Host Address)
        Class: IN (0x0001)
  v Authoritative nameservers
    > com: type NS, class IN, ns e.gtld-servers.net
    > com: type NS, class IN, ns b.gtld-servers.net
    > com: type NS, class IN, ns m.gtld-servers.net
    > com: type NS, class IN, ns j.gtld-servers.net
    > com: type NS, class IN, ns h.gtld-servers.net
    > com: type NS, class IN, ns f.gtld-servers.net
    > com: type NS, class IN, ns g.gtld-servers.net
    > com: type NS, class IN, ns a.gtld-servers.net
    > com: type NS, class IN, ns d.gtld-servers.net
    > com: type NS, class IN, ns i.gtld-servers.net
    > com: type NS, class IN, ns k.gtld-servers.net
    > com: type NS, class IN, ns l.gtld-servers.net
    > com: type NS, class IN, ns c.gtld-servers.net
  v Additional records
    > m.gtld-servers.net: type A, class IN, addr 192.55.83.30
    > l.gtld-servers.net: type A, class IN, addr 192.41.162.30
    > k.gtld-servers.net: type A, class IN, addr 192.52.178.30
    > j.gtld-servers.net: type A, class IN, addr 192.48.79.30
    > i.gtld-servers.net: type A, class IN, addr 192.43.172.30
    > h.gtld-servers.net: type A, class IN, addr 192.54.112.30
    > g.gtld-servers.net: type A, class IN, addr 192.42.93.30
    > f.gtld-servers.net: type A, class IN, addr 192.35.51.30
    > e.gtld-servers.net: type A, class IN, addr 192.12.94.30
    > d.gtld-servers.net: type A, class IN, addr 192.31.80.30
    > c.gtld-servers.net: type A, class IN, addr 192.26.92.30
    > b.gtld-servers.net: type A, class IN, addr 192.33.14.30
    > a.gtld-servers.net: type A, class IN, addr 192.5.6.30
    > m.gtld-servers.net: type AAAA, class IN, addr 2001:501:b1f9::30
    [Request In: 87]
    [Time: 0.173139000 seconds]
```

④ query top-level server and try to find authoritative domain name server

```
Domain Name System (response)
    Transaction ID: 0xc8b5
  > Flags: 0x8000 Standard query response, No error
    Questions: 1
    Answer RRs: 0
    Authority RRs: 5
    Additional RRs: 11
  v Queries
    v www.baidu.com: type A, class IN
          Name: www.baidu.com
          [Name Length: 13]
          [Label Count: 3]
          Type: A (1) (Host Address)
          Class: IN (0x0001)
  v Authoritative nameservers
    > baidu.com: type NS, class IN, ns ns2.baidu.com
    > baidu.com: type NS, class IN, ns ns3.baidu.com
    > baidu.com: type NS, class IN, ns ns4.baidu.com
    > baidu.com: type NS, class IN, ns ns1.baidu.com
    > baidu.com: type NS, class IN, ns ns7.baidu.com
  v Additional records
    > ns2.baidu.com: type A, class IN, addr 220.181.33.31
    > ns2.baidu.com: type AAAA, class IN, addr 240e:940:603:4:0:ff:b01b:589a
    > ns3.baidu.com: type A, class IN, addr 153.3.238.93
    > ns3.baidu.com: type A, class IN, addr 36.155.132.78
    > ns4.baidu.com: type A, class IN, addr 111.45.3.226
    > ns4.baidu.com: type A, class IN, addr 14.215.178.80
    > ns1.baidu.com: type A, class IN, addr 110.242.68.134
    > ns1.baidu.com: type AAAA, class IN, addr 240e:bf:b801:1002:0:ff:b024:26de
    > ns7.baidu.com: type A, class IN, addr 180.76.76.92
    > ns7.baidu.com: type AAAA, class IN, addr 240e:940:603:4:0:ff:b01b:589a
    > ns7.baidu.com: type AAAA, class IN, addr 240e:bf:b801:1002:0:ff:b024:26de
    [Request In: 89]
    [Time: 0.083255000 seconds]
```

⑤ query domain name server, and get a CNAME



⑥ query the domain name server for the CNAME

```
Domain Name System (response)
    Transaction ID: 0xe014
  > Flags: 0x8000 Standard query response, No error
    Questions: 1
    Answer RRs: 0
    Authority RRs: 5
    Additional RRs: 9
  v Queries
    v www.a.shifen.com: type A, class IN
          Name: www.a.shifen.com
          [Name Length: 16]
          [Label Count: 4]
          Type: A (1) (Host Address)
          Class: IN (0x0001)
  v Authoritative nameservers
    > a.shifen.com: type NS, class IN, ns ns4.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns2.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns1.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns3.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns5.a.shifen.com
  v Additional records
    > ns5.a.shifen.com: type A, class IN, addr 180.76.76.95
    > ns4.a.shifen.com: type A, class IN, addr 14.215.177.229
    > ns4.a.shifen.com: type A, class IN, addr 111.20.4.28
    > ns3.a.shifen.com: type A, class IN, addr 36.155.132.12
    > ns3.a.shifen.com: type A, class IN, addr 153.3.238.162
    > ns2.a.shifen.com: type A, class IN, addr 220.181.33.32
    > ns1.a.shifen.com: type A, class IN, addr 110.242.68.42
    > ns5.a.shifen.com: type AAAA, class IN, addr 240e:bf:b801:1006:0:ff:b04f:346b
    > ns5.a.shifen.com: type AAAA, class IN, addr 240e:940:603:a:0:ff:b08d:239d
    [Request In: 93]
    [Time: 0.044080000 seconds]
```

⑦ query the CNAME and get A record

```
Domain Name System (response)
    Transaction ID: 0xf46c
  > Flags: 0x8400 Standard query response, No error
    Questions: 1
    Answer RRs: 2
    Authority RRs: 5
    Additional RRs: 9
  v Queries
    v www.a.shifen.com: type A, class IN
          Name: www.a.shifen.com
          [Name Length: 16]
          [Label Count: 4]
          Type: A (1) (Host Address)
          Class: IN (0x0001)
  v Answers
    > www.a.shifen.com: type A, class IN, addr 182.61.200.108
    > www.a.shifen.com: type A, class IN, addr 182.61.200.110
  v Authoritative nameservers
    > a.shifen.com: type NS, class IN, ns ns4.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns5.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns1.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns2.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns3.a.shifen.com
  v Additional records
    > ns1.a.shifen.com: type A, class IN, addr 110.242.68.42
    > ns2.a.shifen.com: type A, class IN, addr 220.181.33.32
    > ns3.a.shifen.com: type A, class IN, addr 153.3.238.162
    > ns3.a.shifen.com: type A, class IN, addr 36.155.132.12
    > ns4.a.shifen.com: type A, class IN, addr 14.215.177.229
    > ns4.a.shifen.com: type A, class IN, addr 111.20.4.28
    > ns5.a.shifen.com: type A, class IN, addr 180.76.76.95
    > ns5.a.shifen.com: type AAAA, class IN, addr 240e:bf:b801:1006:0:ff:b04f:346b
    > ns5.a.shifen.com: type AAAA, class IN, addr 240e:940:603:a:0:ff:b08d:239d
    [Request In: 95]
    [Time: 0.037204000 seconds]
```